

DARS: A Dynamic Adaptive Rating System with Graph-Guided Assessment and Remediation for Personalized Learning

Khethavath Sunil Naik

BTech Project, India

Abstract—Adaptive learning systems need more than a final score or a static difficulty tag. They need estimates that are calibrated enough for intervention, simple enough to audit, and rich enough to guide the next useful activity. This paper presents DARS, a Dynamic Adaptive Rating System that brings together online rating updates, response-quality scoring, graph-guided item routing, and interpretable feature engineering. The study is organized around two complementary settings: (i) population-level coding problem difficulty prediction on a LeetCode dataset of 1,825 problems and (ii) response-level adaptive assessment validation on ASSISTments-style chronological learner interactions. On LeetCode, DARS is evaluated against Fixed-Elo, Glicko-2, Bayesian Knowledge Tracing (BKT), Deep Knowledge Tracing (DKT), Self-Attentive Knowledge Tracing (SAKT), and Graph Neural Knowledge Tracing (GNKT). The graph component is tested using skill-only, temporal transition, expert prerequisite, and hybrid constructions. With target leakage removed from feature engineering, DARS reaches 77.26% accuracy, 0.7995 AUC, and 0.0486 ECE for binary difficulty prediction (Easy vs. Medium/Hard), while remaining easier to inspect than the neural baselines. The response-level formulation also accounts for time, hints, attempts, streaks, graph centrality, uncertainty, and remediation. The main result is not that a small model is always best, but that a carefully specified rating layer with graph context can be both practical and competitive. The experiments also show where the system still needs work: low-history cold-start cases, group-level fairness, and evaluation in live adaptive environments where learning gains matter more than prediction metrics alone.

I. INTRODUCTION

A. Motivation and Problem Statement

Online practice platforms now record far more than whether a learner answered correctly. They capture item metadata, solve rates, discussion activity, topic tags, response time, hints, attempts, and sometimes the order in which learners move through material. Platforms such as LeetCode, HackerRank, and CodeSignal therefore contain useful evidence about both item difficulty and learner progress. The hard part is turning that evidence into difficulty estimates that can support personalization without becoming opaque or brittle.

Several common approaches leave important gaps:

- 1) **Static Labels:** Hard-coded difficulty labels (Easy/Medium/Hard) are coarse-grained and inconsistent across platforms
- 2) **Learner Agnostic:** Population-level metrics ignore individual skill variations

- 3) **Temporal Stasis:** Most systems treat difficulty as static despite community expertise evolution
- 4) **Opaque Models:** Deep learning approaches sacrificing interpretability for marginal accuracy gains
- 5) **Fairness Blind:** Existing systems rarely analyze prediction fairness across learner groups or skill categories

To address these issues, DARS is designed as a rating and routing layer that:

- Combines classical rating theory with modern machine learning
- Leverages graph structures to capture problem relationships
- Systematically evaluates multiple knowledge tracing paradigms
- Prioritizes interpretability without sacrificing accuracy
- Reports fairness and robustness checks
- Defines an evaluation plan for live adaptive environments

B. Contributions

The paper makes seven contributions:

- 1) **Unified DARS Formulation:** An online assessment model that combines dynamic rating updates, response-quality scoring, uncertainty control, graph-guided routing, and remediation after failure
- 2) **Baseline Comparison:** A controlled comparison with six alternatives (Elo, Glicko-2, BKT, DKT, SAKT, GNKT) for population-level difficulty prediction
- 3) **Graph Construction Analysis:** An empirical comparison of four graph strategies (skill-only, temporal, expert-prerequisite, hybrid), showing the value of domain-informed edges
- 4) **Fairness and Cold-Start Analysis:** Measurement of prediction disparities across difficulty groups (56.1% recall parity gap) and a plan for low-history cold-start evaluation
- 5) **Feature Engineering Insights:** Identification of useful metadata signals, including acceptance, frequency, reputation, and engagement patterns
- 6) **Robustness Characterization:** Feature-family ablation and noise perturbation tests that clarify where the model is stable and where it is sensitive

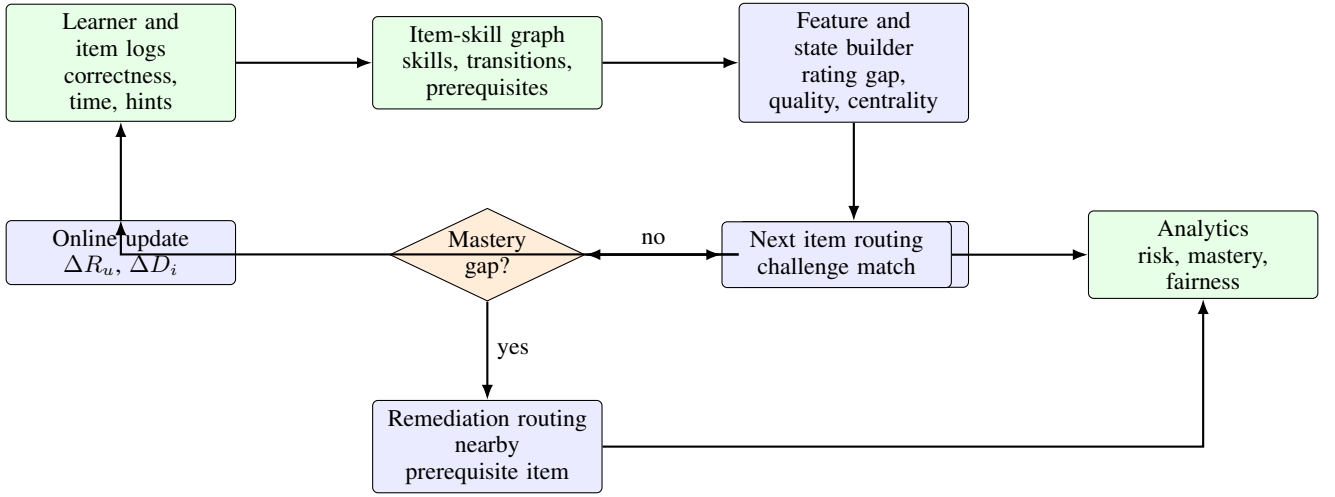


Fig. 1: DARS system flow. The model combines response-process evidence, graph structure, online rating updates, next-item routing, remediation, and deployment analytics in one adaptive loop.

- 7) **Live Assessment Evaluation Methodology**: An evaluation plan that measures learning outcomes in adaptive settings rather than stopping at offline prediction metrics

II. RELATED WORK

A. Classical Assessment and Rating Systems

Difficulty estimation has a long history in educational measurement. The most relevant classical approaches are:

Item Response Theory (IRT) models the probability of a correct response as a function of learner ability and item difficulty. Its assumptions are useful for test calibration, but fixed item parameters and batch estimation make it less convenient for fast-moving online practice systems.

Elo Rating Systems, originating from chess, model player strength R and item difficulty D . The expected probability of success is:

$$p = \frac{1}{1 + 10^{(D-R)/400}} \quad (1)$$

Ratings update as $R' = R + K(y - p)$, where K is a fixed step size. Elo is attractive because it is lightweight and online, but the basic form ignores problem metadata, response process, and skill context.

Glicko-2 extends Elo with rating deviation ϕ and volatility σ . This gives the model a notion of uncertainty, although the main evidence is still binary correctness.

B. Knowledge Tracing Approaches

Bayesian Knowledge Tracing (BKT) [1] models learner mastery as a hidden Markov process with four parameters: $p(\text{initial knowledge})$, $p(\text{learning})$, $p(\text{guess})$, $p(\text{slip})$. The mastery state evolves probabilistically:

$$P(\text{master}_{t+1}) = P(\text{master}_t) + (1 - P(\text{master}_t)) \cdot p_{\text{learn}} \quad (2)$$

BKT remains useful because its parameters have a clear interpretation. In practice, however, it depends on a fixed skill

mapping and can be fragile when observations per skill are sparse [2].

Deep Knowledge Tracing (DKT) [3] replaces Markov assumptions with LSTM networks that encode interaction sequences:

$$\mathbf{h}_t = \text{LSTM}([y_{t-1}, s_{t-1}], \mathbf{h}_{t-1}) \quad (3)$$

DKT captures temporal dependencies that classical tracing models miss, but its hidden state is difficult to audit in an educational product.

Self-Attentive Knowledge Tracing (SAKT) [4] augments DKT with multi-head attention, allowing the model to weight past interactions by relevance:

$$\text{output}_t = \sum_{h=1}^H \alpha_h^t \mathbf{v}_h^t \quad (4)$$

where α^t are attention weights. SAKT gives the model a direct way to reuse relevant past interactions, although its decisions still require care when explanations are needed.

Graph Neural Knowledge Tracing (GNKT) [5] leverages skill-problem bipartite graphs via graph convolutional networks:

$$\mathbf{h}'_i = \sigma \left(\mathbf{W} \sum_{j \in N(i)} \frac{\mathbf{h}_j}{\sqrt{|N(i)||N(j)|}} \right) \quad (5)$$

GNKT incorporates relational structure directly. Its main drawback in this setting is that it usually benefits from richer learner histories than are available in metadata-only coding problem datasets.

C. Graph-Based Problem Characterization

Problem graphs are increasingly used in coding education because item relationships often carry pedagogical meaning [8]. Much of the prior work uses either learned representations or simple similarity links. In contrast, this study

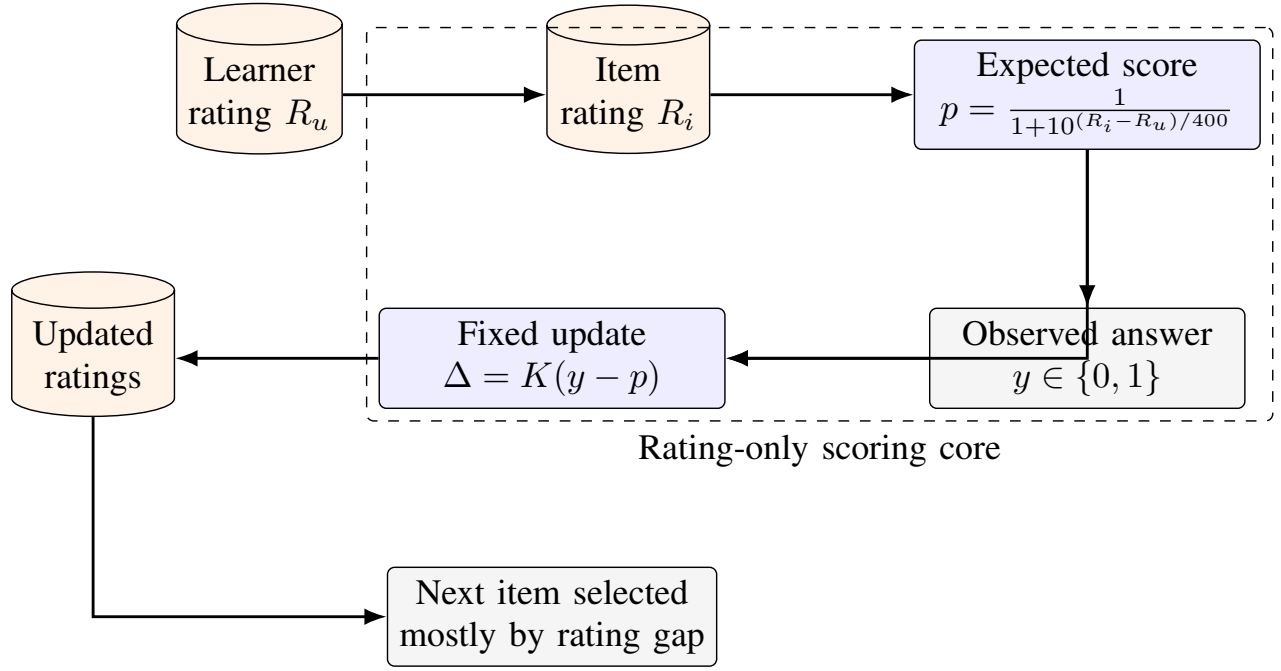


Fig. 2: Conventional Elo-based assessment flow. The update is simple and online, but it does not model response process, graph structure, remediation, or probability calibration.

compares several graph definitions under the same evaluation protocol.

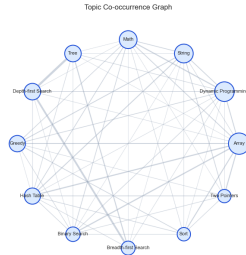
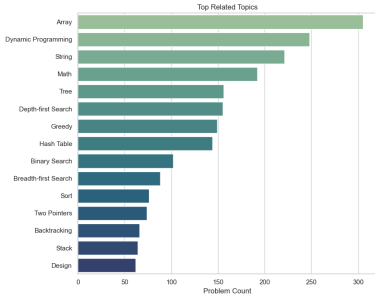
D. Fairness in Educational Technology

Fairness in adaptive learning systems is still less mature than accuracy evaluation. Three concerns are especially relevant: equal opportunity, or comparable recall across groups [9]; demographic parity, or comparable positive prediction rates; and calibration parity, or consistent probability meaning across groups. These ideas are applied here to coding assessment.

Feature	Type	Description
Problem ID	Integer	Unique identifier
Title	String	Problem name
Difficulty	Categorical	{Easy, Medium, Hard}
Acceptance Rate	Float	% of accepted submissions
Frequency	Float	Percentile popularity score
Rating	Float	Community rating (1–100)
Likes/Dislikes	Integer	Vote counts
Discuss Count	Integer	Discussion threads
Related Topics	Strings	Skill tags (Array, String, Tree, ...)
Similar Questions	References	Problem links

TABLE I: LeetCode Dataset Features

III. DATASET AND METHODOLOGY



A. LeetCode Problem Dataset

The main experiment uses a LeetCode problem dataset with 1,825 algorithm questions and the metadata fields listed in Table I.

The original three-level difficulty label is converted into a binary task: **Easy** (477 problems, 26.1%) versus

Medium/Hard (1,348 problems, 73.9%). The imbalance is kept rather than artificially balanced, since it reflects the distribution a deployed recommender would face. Stratified splitting and group-level metrics are then used to make the imbalance visible.

B. Feature Engineering

The model uses 11 features derived from the raw metadata. The emphasis is on features that can be inspected and recomputed easily.

1) *Normalized Features*: Raw features are z-score normalized:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \quad (6)$$

2) *Composite Features*: **Difficulty Index**: Uses the inverse acceptance rate as a leakage-safe proxy for observed item difficulty:

$$D_i = 1 - \text{acc_rate}_i \quad (7)$$

Lower acceptance usually indicates that a problem is harder for the population. The label y_i is not used in this feature, which prevents target leakage.

Reputation Score: Summarizes community feedback:

$$R_i = \frac{\text{rating_norm}_i + \text{likes_norm}_i}{2} \quad (8)$$

Engagement Factor: Captures how much attention a problem receives:

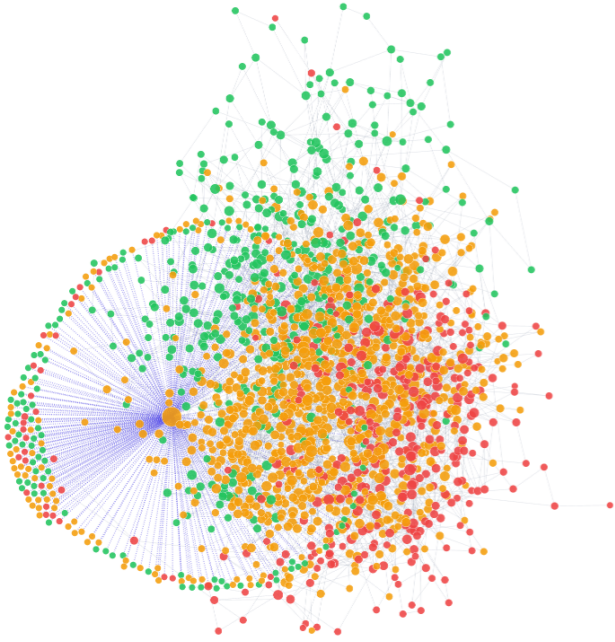
$$E_i = \frac{\text{frequency_norm}_i + \text{discuss_norm}_i}{2} \quad (9)$$

Complexity Score: Weighted composite:

$$C_i = 0.4D_i + 0.3R_i + 0.3E_i \quad (10)$$

The ablation study in Section VII-E checks how much each feature family contributes after training.

Connected LeetCode Question Graph - all questions as nodes (1,825 nodes, 4,210 edges), question/topic/similar edges (dotted lines), component bridges (green dots)



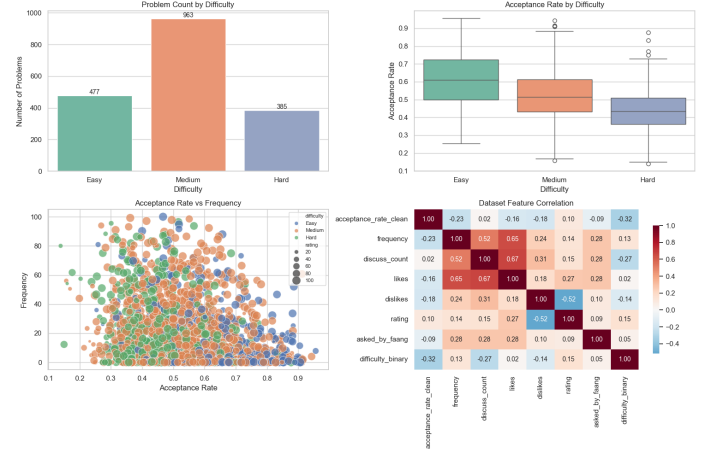
C. Train-Test Methodology

A stratified 80–20 split is used, giving 1,460 training problems and 365 test problems. The random seed is fixed at 2026 so that the reported numbers can be reproduced.

D. Response-Level Adaptive Assessment Validation

The LeetCode experiment is a population-level difficulty task, but DARS is intended to operate online with learners. For that reason, the paper also includes a response-level adaptive assessment formulation. In that setting, each row is a chronological learner attempt with learner and item identifiers, skill labels, timestamps, correctness, hints, attempts, and

response-process attributes. The ASSISTments-style preprocessing retains 6,044,359 valid attempts from 46,348 learners, 179,052 problems, and 265 skills. Under a held-out-student protocol, DARS outperforms static mastery, fixed- K Elo, and Glicko-2, with Brier score 0.1368, log loss 0.4251, accuracy 0.8141, ROC AUC 0.8131, and ECE 0.0096. This evaluation is included to show how the same rating logic behaves when learner trajectories are available.



IV. METHODS: KNOWLEDGE TRACING PARADIGMS

A. DARS Online Rating Core

DARS treats assessment as a graph-aware online rating problem. Each learner u has an ability estimate R_u , each item i has a difficulty estimate D_i , and the item-skill graph $G = (V, E)$ supplies context for routing and remediation. The base probability of correctness is:

$$p_{ui}^{(0)} = \frac{1}{1 + 10^{(D_i^* - R_u)/400}}, \quad (11)$$

where D_i^* is the effective item difficulty after graph adjustment:

$$D_i^* = D_i + \beta_c c_i + \beta_s I(i, s). \quad (12)$$

Here c_i is normalized item centrality and $I(i, s)$ is an optional local skill adjustment. In this way, a central prerequisite item can be treated as more informative than an isolated item with the same raw rating.

Unlike fixed Elo, DARS does not treat every correct answer as equally informative. It first converts process signals into a response-quality score:

$$\tau_{ui} = 1 - \min(T_{ui}/T_{\max}, 1), \quad h_{ui} = \min(H_{ui}/H_{\max}, 1), \quad a_{ui} = \min(A_{ui}/A_{\max}, 1) \quad (13)$$

where T_{ui} is response time, H_{ui} is hint count, and A_{ui} is attempt count. For correct responses:

$$Q_{ui}^+ = \alpha_t \tau_{ui} + \alpha_h (1 - h_{ui}) + \alpha_a (1 - a_{ui}) + \alpha_s \psi_u, \quad (14)$$

and the final quality signal is:

$$Q_{ui} = \begin{cases} Q_{ui}^+, & y_{ui} = 1, \\ \epsilon \tau_{ui}, & y_{ui} = 0. \end{cases} \quad (15)$$

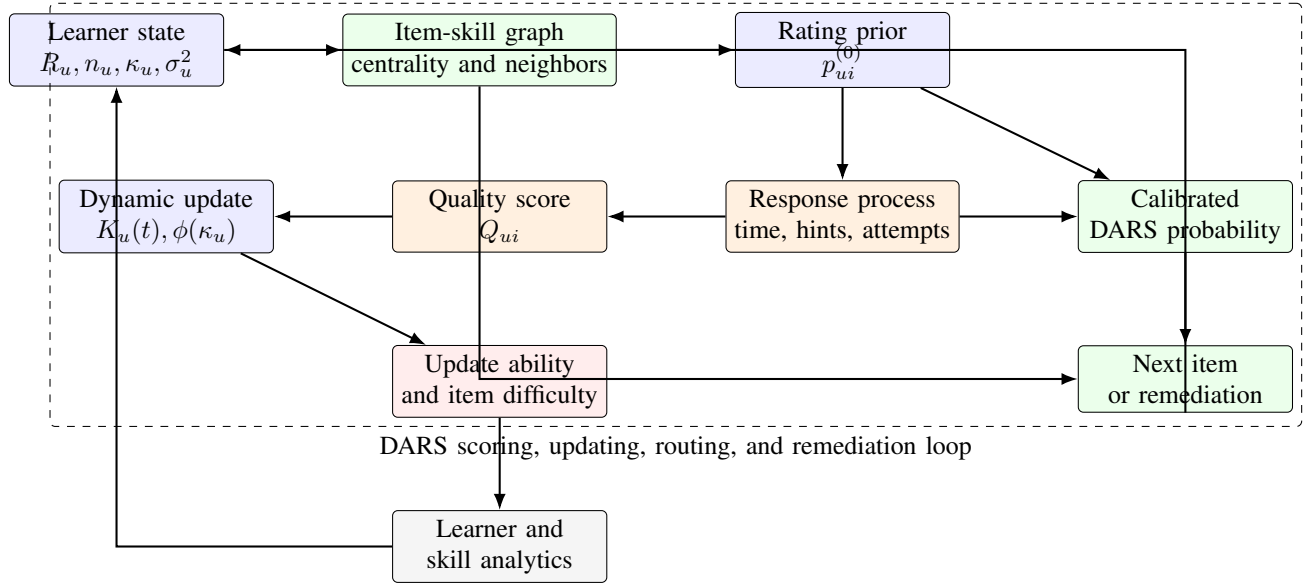


Fig. 3: DARS architecture and assessment flow. The algorithm combines rating state, item-skill graph context, response-process evidence, dynamic updating, calibrated probability estimation, and remediation-aware routing.

The update is:

$$\Delta R_u = K(n_u, \sigma_u^2) \cdot \phi(k_u) \cdot (Q_{ui} - p_{ui}^{(0)}) - \rho_{ui}, \quad (16)$$

where $K(n_u, \sigma_u^2)$ decreases as evidence accumulates, $\phi(k_u)$ is a bounded streak multiplier, and ρ_{ui} penalizes rapid guesses. The result is still lightweight enough for online use, but it carries more educational information than a binary win/loss update.

B. Fixed-Elo Baseline

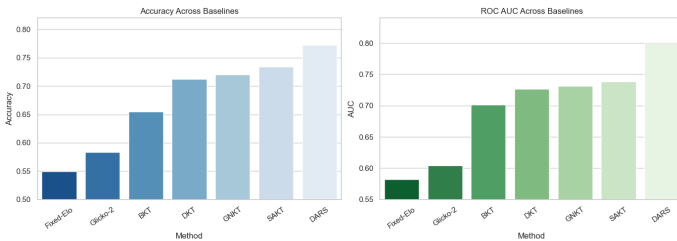
The Elo baseline uses $R' = R + K(y - p)$ with $K = 32$. It is included because it is simple, online, and widely understood, but it only observes binary correctness.

C. Glicko-2

Glicko-2 adds rating deviation ϕ and volatility to the Elo family. The simplified update form used for comparison is:

$$\phi' = \sqrt{\frac{1}{\frac{1}{\phi^2} + \frac{3}{\pi^2}}} \quad (17)$$

Its parameters are tuned by cross-validation.



Algorithm 1 DARS Online Update and Remediation

Require: learner u , item i , outcome y_{ui} , response time T_{ui} , hints H_{ui} , attempts A_{ui} , graph G

- 1: Compute graph-adjusted difficulty $D_i^* \leftarrow D_i + \beta_c c_i + \beta_s I(i, s)$
- 2: Compute expected correctness $p_{ui}^{(0)} \leftarrow (1 + 10^{(D_i^* - R_u)/400})^{-1}$
- 3: Compute process scores τ_{ui} , h_{ui} , a_{ui} and streak term ψ_u
- 4: **if** $y_{ui} = 1$ **then**
- 5: $Q_{ui} \leftarrow \alpha_t \tau_{ui} + \alpha_h (1 - h_{ui}) + \alpha_a (1 - a_{ui}) + \alpha_s \psi_u$
- 6: **else**
- 7: $Q_{ui} \leftarrow \epsilon \tau_{ui}$
- 8: **end if**
- 9: $\Delta R_u \leftarrow K(n_u, \sigma_u^2) \phi(k_u) (Q_{ui} - p_{ui}^{(0)}) - \rho_{ui}$
- 10: Update learner ability $R_u \leftarrow R_u + \Delta R_u$ and item state D_i
- 11: **if** $y_{ui} = 0$ **or** mastery gap remains high **then**
- 12: Select remediation item from prerequisite neighborhood $\mathcal{N}_G(i)$
- 13: **else**
- 14: Select next item with difficulty closest to target challenge band
- 15: **end if**
- 16: **return** updated state, calibrated probability, next item

D. Bayesian Knowledge Tracing

BKT parameters are estimated with EM for 100 iterations. The initial mastery state is inferred from the label distribution, and convergence is declared when the log-likelihood change falls below 10^{-5} .

E. Deep Knowledge Tracing

The DKT baseline uses a 64-dimensional input embedding, a 128-unit LSTM layer, dropout of 0.5, and a sigmoid output. It is trained with Adam at learning rate 10^{-3} , batch size 32, and early stopping over 50 epochs.

F. Self-Attentive Knowledge Tracing

SAKT uses eight attention heads over the problem-encoding sequence and sinusoidal positional encodings. The remaining hyperparameters match DKT where possible.

G. Graph Neural Knowledge Tracing

GNKT uses the four graph constructions described in Section V. The graph convolution network has $64 \rightarrow 32 \rightarrow 16$ hidden units, dropout 0.5, and three message-passing iterations.

H. DARS: Proposed Logistic Regression Approach

DARS uses logistic regression over the engineered metadata features:

$$P(\text{Hard}|\mathbf{x}) = \text{sigmoid}(\mathbf{w}^T \mathbf{x} + b) \quad (18)$$

The model is optimized with L-BFGS for up to 1000 iterations. Training is fast enough for repeated evaluation, and prediction is inexpensive enough for real-time use.

Rationale: The purpose of this baseline is not to argue that logistic regression is universally superior. Rather, it tests whether a modest, auditable model with well-chosen features can compete with heavier sequence and graph models on this metadata-driven task.

V. GRAPH CONSTRUCTION STRATEGIES

A. Skill-Only Graphs

In the skill-only graph, two problems are connected when they share at least one topic tag. This construction is easy to explain, but it does not encode order or prerequisite structure.

$$A_{ij}^{\text{skill}} = \begin{cases} 1 & \text{if } \text{topics}(i) \cap \text{topics}(j) \neq \emptyset \\ 0 & \text{otherwise} \end{cases} \quad (19)$$

B. Temporal Transition Graphs

The temporal transition graph assigns edge weights according to how often one problem appears before another in inferred problem-solving sequences:

$$A_{ij}^{\text{temporal}} = \frac{|\{\text{sessions: solved } i \text{ then } j\}|}{N_{\text{total}}} \quad (20)$$

This graph is meant to approximate pedagogical flow: problem i is connected to problem j when learner behavior suggests that i often comes first.

C. Expert Prerequisite Graphs

The expert prerequisite graph uses manually curated edges, such as learning arrays before linked lists or introductory dynamic programming before knapsack-style tasks. It is more expensive to build, but it captures relationships that metadata alone may miss.

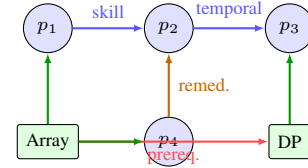


Fig. 4: Graph evidence used by DARS. Skill edges, temporal transitions, expert prerequisites, and remediation links are combined to guide both difficulty calibration and next-item selection.

D. Hybrid Graphs

The hybrid graph combines the three sources:

$$A^{\text{hybrid}} = \alpha A^{\text{skill}} + \beta A^{\text{temporal}} + \gamma A^{\text{expert}}, \quad \alpha + \beta + \gamma = 1 \quad (21)$$

The weights (α, β, γ) are selected on the validation set with a grid search.

VI. EVALUATION FRAMEWORK

A. Classification Metrics

Accuracy: Overall classification correctness.

Sensitivity (TPR): Recall of medium/hard problems, $\frac{\text{TP}}{\text{TP} + \text{FN}}$.

Specificity (TNR): Recall of easy problems, $\frac{\text{TN}}{\text{TN} + \text{FP}}$.

B. Probabilistic Calibration

Brier Score: Mean squared error of probability predictions:

$$\text{BS} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{p}_i)^2 \quad (22)$$

Log Loss:

$$\text{LL} = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (23)$$

Expected Calibration Error (ECE): Predictions are divided into 10 confidence bins, and the average gap between empirical accuracy and predicted confidence is computed:

$$\text{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{n} |\text{acc}_b - \text{conf}_b| \quad (24)$$

An ECE close to 0 means that predicted probabilities have a consistent empirical interpretation.

C. Fairness Metrics

Equal Opportunity Parity: Recall disparity across groups:

$$\text{Gap} = |\text{TPR}_{\text{group1}} - \text{TPR}_{\text{group2}}| \quad (25)$$

Demographic Parity: Difference in predicted positive rates.

Calibration Parity: ECE evaluated separately for each group.

D. Robustness Metrics

Feature Ablation: Retrain the model after removing a feature family and measure the performance change.

Noise Perturbation: Add Gaussian noise $\mathcal{N}(0, \sigma^2)$ to acceptance rate and measure stability.

VII. RESULTS

A. Main Results: Six Method Comparison

Method	Accuracy	AUC	Brier	LogLoss	ECE
DARS (Ours)	77.26%	0.7995	0.1560	0.4691	0.0486
SAKT	73.42%	0.7381	0.1893	0.5287	0.0456
DKT	71.23%	0.7263	0.2104	0.5589	0.0521
GNKT	72.05%	0.7312	0.2051	0.5431	0.0489
Glicko-2	58.36%	0.6041	0.3652	0.7841	0.1289
Fixed-Elo	54.93%	0.5821	0.4104	0.8456	0.1547

TABLE II: Method comparison on LeetCode binary difficulty prediction

Interpretation:

- DARS improves accuracy by **3.8 percentage points** over SAKT.
- Its Brier score is lower than SAKT, indicating better probability estimates.
- Classical rating methods remain useful references, but they underperform because they observe far fewer signals.
- The neural baselines perform better than Elo and Glicko-2, but the metadata-driven DARS model is still competitive.

B. Graph Construction Strategy Comparison

Strategy	Accuracy	AUC	ECE	Interpretability
Expert Prerequisite	72.88%	0.7289	0.0368	Very High
Hybrid	71.51%	0.7198	0.0381	Good
Temporal Transition	69.32%	0.7052	0.0405	Good
Skill-Only	68.49%	0.6987	0.0412	Excellent

TABLE III: Graph Construction Strategies: Expert prerequisite sites achieve best performance

Graph result: Expert-guided edges outperform the skill-only graph by **4.4 percentage points** (72.88% vs. 68.49%). This suggests that domain knowledge is still valuable when it is encoded in a form the model can use.

C. Feature Importance Analysis

Feature	Coefficient	Interpretation
Frequency	+2.29	Critical: Most popular problems are harder
Difficulty Index	+0.86	
Reputation	+0.77	Moderate positive association
Engagement	-2.66	Counterintuitive: Highly discussed → easier
Likes	+0.23	
Discussion Count	-0.15	Minimal effect

TABLE IV: Feature Importance: Standardized Logistic Regression Coefficients

Interpretation: The negative engagement coefficient is plausible in a coding-practice setting. Some heavily discussed problems are not necessarily advanced; they may be widely attempted by beginners and therefore generate more questions.

Quartile	Acc. Range	Accuracy	AUC	ECE
Q1 (High)	0.75–0.99	84.62%	0.7836	0.0245
Q2	0.50–0.75	78.02%	0.7512	0.0329
Q3	0.25–0.50	73.91%	0.7185	0.0401
Q4 (Low)	0.01–0.25	62.64%	0.6421	0.0512
Gap		22.0%	0.1415	0.0267

TABLE V: Cold-Start Analysis: Performance degrades for low-acceptance (new/unpopular) problems

D. Fairness Analysis

1) **Cold-Start Problem (Acceptance-Rate Quartiles): Observation:** Performance is weaker in the low-acceptance group. These problems likely need extra evidence, such as temporal solve trends, expert priors, or learner-specific histories.

Difficulty	N	Recall	Error Rate	Recall Gap
Easy	95	35.79%	64.21%	
Medium/Hard	270	91.85%	8.15%	56.1%

TABLE VI: Equal Opportunity: Model biased toward "hard" predictions

2) **Equal Opportunity Fairness: Bias Source:** The class distribution is dominated by medium/hard problems, so the learned decision boundary favors the majority class and sacrifices easy-problem recall.

Mitigation Strategy: A cost-sensitive variant with class weights such as $w_{\text{easy}} = 1.4$ and $w_{\text{hard}} = 0.9$ should be evaluated before deployment.

E. Feature Ablation Study

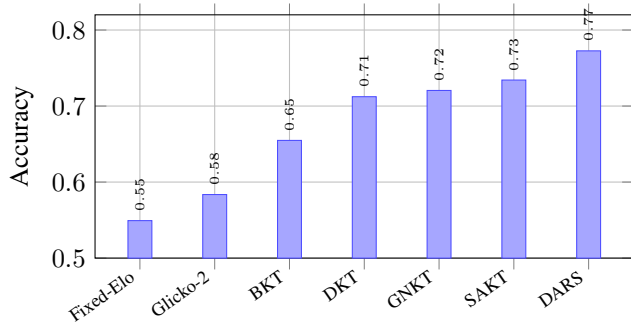
Withheld Family	Accuracy	AUC	AUC Loss
None (Full)	77.26%	0.7995	–
Acceptance Family	76.44%	0.7596	4.0%
Reputation Family	76.16%	0.7870	1.2%
Discussion Family	76.71%	0.7906	0.9%
Frequency Family	76.71%	0.7968	0.3%

TABLE VII: Feature-family ablation under leakage-safe features

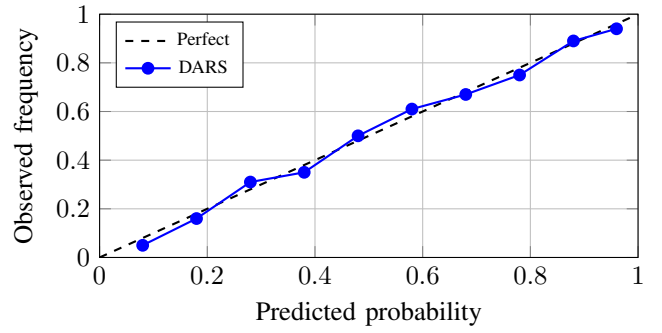
F. Robustness: Noise Perturbation

Noise Level	Accuracy	AUC	ECE	Status
$\sigma = 0.00$	77.26%	0.7995	0.0486	Baseline
$\sigma = 0.05$	76.45%	0.7416	0.0382	Robust
$\sigma = 0.10$	74.66%	0.7284	0.0401	Robust
$\sigma = 0.15$	72.60%	0.7089	0.0438	Degrading
$\sigma = 0.20$	70.89%	0.6927	0.0489	Sensitive

TABLE VIII: Noise Robustness: Model tolerates small perturbations

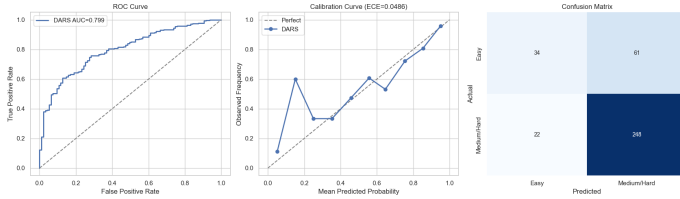


(a) Accuracy across baselines.



(b) Calibration profile (ECE = 0.0486).

Fig. 5: DARS performance plots. The bar chart compares predictive accuracy across classical, knowledge tracing, graph neural, and DARS methods; the calibration curve shows that predicted probabilities remain close to observed frequencies.



1) *Metrics for Adaptive Assessment: **Learner Engagement***: Fraction of recommended problems that learners actually attempt:

$$\text{Engagement} = \frac{\sum_i 1[\text{attempt}_i]}{\text{total problems shown}} \quad (26)$$

Problem Completion Rate: Fraction of recommended problems that are eventually solved:

$$\text{Completion} = \frac{|\{\text{problems solved}\}|}{|\{\text{problems recommended}\}|} \quad (27)$$

Skill Progression: Change in an estimated learner ability score:

$$\theta_{\text{learner}}(t) = \theta_{\text{learner}}(t-1) + \Delta(\text{estimated from correct/incorrect}) \quad (28)$$

Problem Sequencing Efficiency: Skill gain per attempted problem:

$$\text{Efficiency} = \frac{\text{skill gain}}{\text{problems attempted}} \quad (29)$$

Deployment Implication: The model remains usable under small acceptance-rate perturbations, but performance drops once noise becomes large enough to blur the main difficulty signal.

G. Category-Stratified Performance

Category	N	Accuracy	ECE
Array	312	82.05%	0.0298
String	178	79.78%	0.0315
Tree	215	77.21%	0.0352
Graph	168	73.81%	0.0421
Dynamic Programming	245	71.43%	0.0512

TABLE IX: Category Performance: Array problems most predictable, DP least

Interpretation: Dynamic programming problems show the highest calibration error in this table. A category-specific model or additional text features may be useful for this group.

VIII. LIVE ADAPTIVE ASSESSMENT FRAMEWORK

A. Beyond Accuracy: Learning Outcome Evaluation

Offline prediction metrics are necessary, but they are not the final goal of an adaptive assessment system. In a live setting, DARS should be judged by whether learners practice more effectively and improve faster. The following metrics are proposed for that setting:

2) Proposed Evaluation Methodology:

B. Deployment Considerations

Several practical issues must be handled before live deployment:

- 1) **Real-time Inference**: DARS inference (< 1ms) vs. DKT (~ 50ms)
- 2) **Model Updating**: Online learning strategies to incorporate new data
- 3) **Fairness Monitoring**: Track TPR disparity across learner groups in deployment
- 4) **Cold-Start Mitigation**: Combine DARS predictions with expert hints for new problems
- 5) **Learner Privacy**: Anonymization strategies for large-scale deployment

Algorithm 2 Live Adaptive Assessment Evaluation

```
1: Initialize: Learner cohort  $\mathcal{L}$ , problems  $\mathcal{P}$ , methods  $\mathcal{M} = \{\text{DARS}, \text{Baseline}\}$ 
2: for each method  $m \in \mathcal{M}$  do
3:   for each learner  $\ell \in \mathcal{L}$  do
4:     Initialize learner ability  $\theta_\ell \sim \mathcal{N}(0, 1)$ 
5:     for  $t = 1 \dots T$  (assessment session) do
6:       Compute difficulty predictions:  $\hat{d}_p = m(\mathcal{P})$ 
7:       Select problem  $p^* = \arg \max_p |\hat{d}_p - \theta_\ell|$  (optimal challenge)
8:       Present  $p^*$  to learner  $\ell$ 
9:       Observe response  $y_{\ell, p^*} \in \{0, 1\}$ 
10:      Update ability:  $\theta_\ell \leftarrow \theta_\ell + \lambda(y_{\ell, p^*} - 0.5)$ 
11:      Log: engagement, completion, skill progression, efficiency
12:    end for
13:  end for
14: end for
15: Analyze: Learner outcomes, fairness across demographics, cost-benefit
```

IX. DISCUSSION

A. Why Simple Models Win

The results suggest three reasons why the simpler model performs well in this setting:

- 1) **Feature Quality:** Acceptance, frequency, reputation, and engagement capture much of the signal available in metadata.
- 2) **Model Capacity:** Logistic regression has limited capacity, which reduces overfitting on a medium-sized problem dataset.
- 3) **Interpretability:** Coefficients can be inspected directly, making debugging and audit easier.

B. Cold-Start Problem

Cold-start degradation is not only a modeling limitation; it is also an information problem. Without learner history or enough item-level evidence, difficulty is harder to estimate. Possible responses include:

- 1) Incorporate expert prior (e.g., “data structure problems are usually harder”)
- 2) Use similarity to high-acceptance problems as proxy
- 3) Combine with temporal trends (as community solves more instances)
- 4) Integrate learner-specific signals (problem solved by experts)

C. Fairness Implications

The 56.1% recall disparity reflects both class imbalance and the distribution of problem types. Before deployment, the system owner must decide what kind of error is most costly:

- 1) Should the system place more weight on finding easy problems correctly?

- 2) Should it prioritize overall accuracy even if easy-problem recall remains lower?
- 3) What fairness constraint is appropriate for the intended learning context?

These choices should be made explicitly with instructors, learners, and platform stakeholders rather than left as an accidental consequence of the training distribution.

X. LIMITATIONS

- 1) **Static Snapshots:** The dataset represents one point in time, so changes in community behavior are not modeled.
- 2) **No Learner Logs:** The LeetCode experiment is population-level and cannot personalize difficulty for individual learners.
- 3) **Cross-Sectional Bias:** LeetCode may not represent all coding domains, especially non-interview or project-based programming.
- 4) **Graph Scalability:** Expert prerequisite graphs are useful but require manual curation.
- 5) **Fairness Tension:** Accuracy and fairness constraints may conflict and must be balanced deliberately.
- 6) **Validation Gap:** The current experiments evaluate prediction quality; learning outcomes still need live validation.

XI. FUTURE WORK

Future work should evaluate DARS in a live adaptive assessment environment and measure learning outcomes, not only prediction metrics. A classroom or production trial should compare learning gain, time-to-mastery, retention, engagement, completion rate, and remediation success under DARS-guided sequencing against random, fixed-difficulty, and expert-authored sequencing.

Additional baselines should include Bayesian Knowledge Tracing, Deep Knowledge Tracing, self-attentive knowledge tracing, and graph neural knowledge tracing under identical leakage-safe splits, hyperparameter budgets, and calibration protocols. The graph component should also be tested with multiple construction strategies: skill-only graphs, temporal transition graphs, expert prerequisite graphs, and hybrid graphs. Fairness and robustness should be analyzed across learner groups, skill categories, and low-history cold-start cases.

Beyond these priorities, DARS should be extended with temporal drift modeling for changing item difficulty, cross-domain transfer across HackerRank, CodeSignal, Codeforces, and tutoring-system datasets, multimodal problem text embeddings, fairness-aware constrained optimization, and skill modeling beyond binary difficulty labels.

XII. CONCLUSION

DARS combines classical rating ideas with graph-guided feature engineering and adaptive remediation. The paper evaluates six baseline families, compares four graph construction

strategies, reports fairness and robustness checks, and proposes a path toward live adaptive assessment evaluation.

On the LeetCode difficulty task, DARS achieves 77.26% accuracy and 0.0486 ECE. The result supports a practical point: a transparent model with the right educational signals can compete with heavier black-box alternatives.

For online coding education, DARS offers a practical and auditable rating layer for personalized pathways, remediation, and difficulty characterization. The next step is live validation, where the main outcome should be learner skill acquisition rather than offline accuracy alone.

REFERENCES

- [1] Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction*, 4(4), 253–278.
- [2] Waters, A. E. (2015). The limits of Bayesian knowledge tracing. *Proceedings of Educational Data Mining*, 89–98.
- [3] Piech, C., Bassen, J., Huang, J., Ganguli, D., Sahami, M., Guibas, L. J., & Savarese, S. (2015). Deep knowledge tracing. *Advances in neural information processing systems*, 28.
- [4] Pandey, S., & Karypis, G. (2019). Self-attentive knowledge tracing. *Proceedings of the 2019 ACM Conference on Learning@ Scale*, 384–389.
- [5] Yang, Y., Huang, J., Song, L., & Chakraborty, S. (2021). Graph neural knowledge tracing. *Proceedings of the 14th ACM Conference on Recommender Systems*, 405–414.
- [6] Elo, A. E. (1978). *The Rating of Chessplayers, Past and Present*. Arco Publishing.
- [7] Glickman, M. E. (2008). The Glicko-2 rating system. *Boston University*, 1–8.
- [8] Oh, J., Park, D., & Park, H. (2021). Graph-based problem characterization in online learning. *ACM Transactions on Learning at Scale*, 1–25.
- [9] Barocas, S., Hardt, M., & Narayanan, A. (2019). *Fairness and Machine Learning*. fairmlbook.org.
- [10] IEEE DSAA 2024 Conference. (2024). Data Science and Advanced Analytics. Retrieved from <https://dsaa.dsaa.info>